

第 08 讲 内存分配与内存映射

一、内存分配

➤ 内核态内存分配

■ 用户态内存空间分配

- ◆ malloc/free
 - 虚拟地址空间
 - 返回的是线性地址
 - 在驱动程序中不能使用

■ 内核态内存空间分配

- ◆ kmalloc
- ◆ vmalloc
- ◆ get_free_page

➤ kmalloc

Kmalloc 内存分配引擎是一个功能强大的工具，它分配的区域在物理内存中也是连续的。

函数原型

```
#include <linux/slab.h>
```

```
void *kmalloc(size_t size, int flags);
```

size —— 要分配的块的大小
flags —— 分配标志

➤ size 参数

内核负责管理物理内存，物理内存只能按页面进行分配；

Linux 处理内存分配的方法是，创建一系列内存对象池，每个池中的内存块大小是固定一致的。处理分配请求时就直接在包含有足够大的内存池中传递一整块给请求者；

size 的大小一般是 2 的 n 次方，kmalloc 能分配的最小内存块是 32 或 64；

kmalloc 能分配的内存块大小存在一个上限，为了保证代码的可移植性，不应利用 kmalloc 分配超过 128KB 的内存。

➤ flags 参数

最常用的 flags 参数是 **GFP_KERNEL**，表示内存分配是运行在内核空间的进程执行的；

- GFP_前缀意味着调用 get_free_pages 来实现实际的内存分配

kmalloc 允许空闲内存不足时进程进入休眠状态，因此使用 GFP_KERNEL 的函数必须是可重入的；

GFP_KERNEL 标志不能在中断处理例程中调用；

❑ GFP_ATOMIC

- 用于在中断处理例程或其它运行于进程上下文之外的代码中分配内存，不会休眠

❑ GFP_KERNEL

- 内核内存的通常分配方法，可能会休眠

➤ __GFP_前缀的标志可与 GFP_前缀的标志“或”起来使用

❑ `_GFP_DMA`

- 该标志请求分配发生在可进行 DMA 的内存区段中

❑ `_GFP_HIGHMEM`

- 该标志表明要分配的内存可位于高端内存

➤ Linux 内核把内存分为三个区段：可用于 DMA 的内存、常规内存、高端内存。

➤ kfree

`kmalloc` 申请内存地址空间由 `kfree` 进行释放

函数原型

```
void kfree(void *obj);
```

➤ vmalloc

- `vmalloc` 分配虚拟地址空间的连续区域，但不保证物理地址空间的连续性
- `vmalloc` 分配的地址空间在 `VMALLOC_START` 到 `VMALLOC_END` 的范围中
- `vmalloc` 不能在原子上下文中使用，可能会引起休眠
- `vmalloc` 申请的地址空间由 `vfree` 进行释放

函数原型

```
#include <linux/vmalloc.h>
void *vmalloc(unsigned long size);
void vfree(void *addr);
```

➤ get_free_page

如果模块需要分配大块的内存，更加适合使用面向页的分配技术

The diagram shows four functions in a light blue box, each with a green callout box pointing to it:

- `get_zeroed_page(unsigned int flags);` → 返回指向新页面的指针并将页面清零
- `__get_free_page(unsigned int flags);` → 类似 `get_zeroed_page`，但页面不清零
- `__get_free_pages(unsigned int flags, unsigned int order);` → 分配若干物理连续的页面，不清零页面
- `__get_free_pages(unsigned int flags, unsigned int order);` (Note: The diagram shows this function twice)

❑ 参数 `flags` 的作用和 `kmalloc` 中的一样

❑ 参数 `order` 是要申请的页面数的以 2 为底的对数。例如 `order` 为 3 时表示要申请 8 个页面。

❑ 可允许的 `order` 最大值是 10 或 11

★ 当程序不再使用页面时，可使用下列函数进行释放：

```
void free_page(unsigned long addr);
```

```
void free_pages(unsigned long addr, unsigned int order);
```

❑ 如果试图释放和先前分配数目不等的页面，内存映射关系会被破坏，系统会出错。

➤ kmalloc vs. vmalloc vs. get_free_page

三者返回的都是虚拟地址，但是

- `kmalloc` 和 `get_free_page` 申请的内存位于物理内存映射区域，在物理上是连续的，`virt_to_phys()`可

以实现内核虚拟地址转化为物理地址;

- `vmalloc` 申请的内存则位于 `vmalloc_start ~ vmalloc_end` 之间, 与物理地址没有简单的转换关系, 虽然在逻辑上是连续的, 但是在物理上不要求连续。
- `kmalloc()` 分配连续的物理地址, 用于小内存分配
- `__get_free_page()` 分配连续的物理地址, 用于整页分配
- `vmalloc` 分配得到的地址不能在微处理器之外使用, 因为它们只在处理器 MMU 上才有意义

➤ 后备高速缓存

设备驱动程序常常会反复地分配很多同一大小的内存块

后备高速缓存 (lookaside cache), linux 内核又称为 `slab` 分配器

`cat /proc/slabinfo` 查看后备高速缓存的统计信息

- slab维护的后备高速缓存是`kmem_cache_t`类型, 可以由`kmem_cache_create`函数创建一个可以容纳任意数目的内存区域, 区域大小相同

```
kmem_cache_t *kmem_cache_create(const char *name, size_t size,
                                size_t offset,
                                unsigned long flags,
                                void (*constructor)(void *, kmem_cache_t *, unsigned long flags),
                                void (*destructor)(void *, kmem_cache_t *, unsigned long flags));
```

- `size`指定区域大小
- `offset`指定页面中第一个对象的偏移量, 常设置为0
- `flags`
 - `SLAB_HWCACHE_ALIGN`—要求数据对象与高速缓存行对齐
 - `SLAB_CACHE_DMA`—要求数据对象从DMA内存段中分配

- 使用`kmem_cache_create`创建了某个对象的后备高速缓存后, 就可以调用`kmem_cache_alloc`从中分配内存对象

```
void *kmem_cache_alloc(kmem_cache_t *cache, int flags);
```

- 释放一个内存对象使用`kmem_cache_free`函数

```
void kmem_cache_free(kmem_cache_t *cache, const void *obj);
```

- 如果驱动程序不会再使用后备高速缓存了, 应该调用`kmem_cache_destroy`函数释放后备高速缓存

```
int kmem_cache_destroy(kmem_cache_t *cache);
```

➤ 内存池

内存池(Memory Pool)技术是在真正使用内存之前, 先申请分配一定数量的、大小相等(一般情况下)的内存块留作备用。当有新的内存需求时, 就从内存池中分出一部分内存块, 若内存块不够再继续申请新的内存。

➤ 内存池的创建函数 `mempool_create` 的函数原型如下:

```
mempool_t *mempool_create(int min_nr,  
                           mempool_alloc_t *alloc_fn,  
                           mempool_free_t *free_fn,  
                           void *pool_data)
```

- ❑ `min_nr` 指定内存池应该始终保持的已分配对象的最小数目
- ❑ `alloc_fn` 和 `free_fn` 为对象实际分配和释放函数
- ❑ `pool_data` 为传入 `alloc_fn` 和 `free_fn` 的参数

```
typedef void *(mempool_alloc_t)(int gfp_mask, void *pool_data);  
typedef void (mempool_free_t)(void *element, void *pool_data);
```

➤ 在创建内存池之后, 可调用如下函数分配和释放对象

```
void *mempool_alloc(mempool_t *pool, int gfp_mask)  
void mempool_free(void *element, mempool_t *pool)
```

➤ 可利用 `mempool_resize` 函数来调整 `mempool` 的大小

```
int mempool_resize(mempool_t *pool, int new_min_nr, int gfp_mask);
```

➤ 如果不再使用内存池, 需销毁内存池

```
void mempool_destroy(mempool_t *pool);
```

- ❑ 在销毁内存池之前, 必须将所有已分配的对象返回到内存池中, 否则会导致内核oops

➤ 后备高速缓存 VS. 内存池

- 内核中经常需要申请大量的相同类型的较小数据结构的内存, 例如 `skb`、`urb`, 这样容易出现内存碎片
- 使用 **SLAB** 预先分配一片空间专门用来给这类数据结构使用, 能够提高申请和回收速度, 减少内存碎片
- **SLAB** 适合于大量的、细小的数据结构的内存申请的情况
- 内存池, 大多用于块设备, 如文件系统
- 在处理大量的数据时, 可能内核可用的内存不足, 使用内存池预先申请一块内存
- 这样使用内存池 API `mempool_alloc`, 首先尝试 `alloc_fn`, 如果失败就从内存池中获取预分配的内存, 能够保证 `mempool_alloc` 一定申请成功, 不会陷入睡眠 (当然预分配的内存没有了, 也会睡眠)。

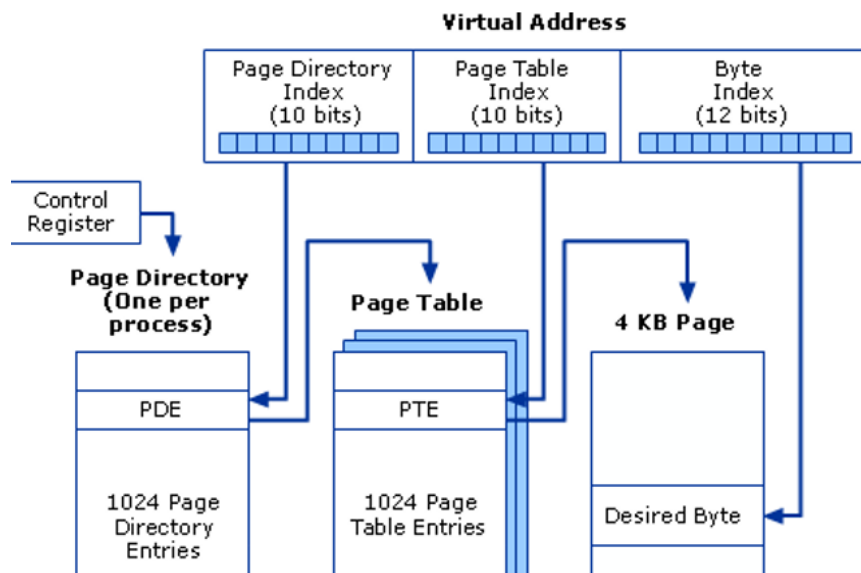
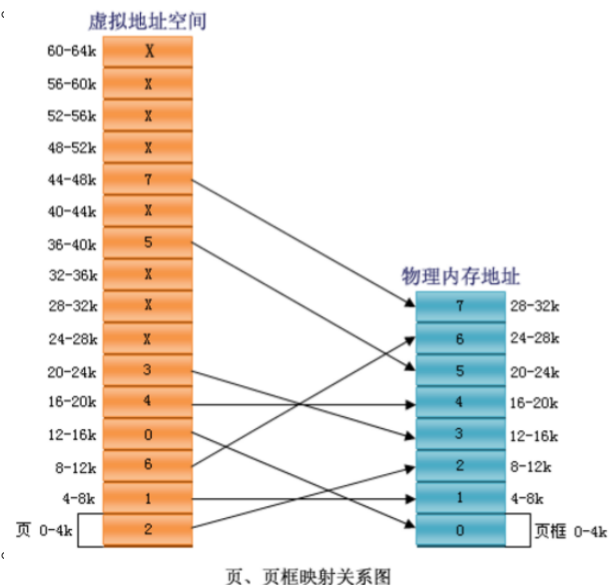
二、 内存映射

➤ 存储管理单元 MMU

- MMU (Memory Manage Unit, 存储管理单元) 在 CPU 和物理内存之间进行地址转换, 将地址从逻辑空间映射到物理空间, 这个转换过程一般称为内存映射。
- MMU 主要完成以下工作:
 - (1) 虚拟存储空间到物理存储空间的映射。
 - (2) 存储器访问权限的控制。
 - (3) 设置虚拟存储空间的缓冲的特性。

➤ 页表

- 嵌入式系统中常常采用页式存储管理。
- 页表是存储在内存中的一个表, 页表用来管理这些页。
- 页表的每一行对应于虚拟存储空间的一页, 该行包含了该虚拟内存页对应的物理内存页的地址、该页的方位权限和该页的缓冲特性等。
- 从虚拟地址到物理地址的变换过程就是查询页表的过程。



➤ 内存映射和页结构

- page结构体, 对系统中每个物理页, 都有一个page结构相对应。

```
#include <linux/mm.h>
struct page {
    atomic_t count; //对该页的访问计数, 计数为0时该页返回给空闲链表
    void *virtual; //如果页面被映射, 则指向页的内核虚拟地址; 否则为NULL
    unsigned long flags; //描述页状态的一系列标志
    ... ..
}
```

- 内核维护了一个或多个page结构数组, 用于跟踪系统中的物理内存。

- 内核有一些函数和宏用于在page结构指针与虚拟地址之间进行转换。

```
struct page *virt_to_page(void *kaddr);
```

该宏在<asm/page.h>中定义，负责将内核逻辑地址转换为响应的page结构指针。

```
struct page *pfn_to_page(int pfn);
```

针对给定的页帧号，返回page结构指针。

```
void *page_address(struct page *page);
```

如果地址存在的话，则返回页的内核虚拟地址。

```
struct page *vmalloc_to_page(const void *addr);
```

```
unsigned long vmalloc_to_pfn(const void *addr);
```

vmalloc申请的虚拟地址到page和pfn的映射

➤ 虚拟内存区

虚拟内存区 (VMA) 用于管理进程地址空间中不同区域的内核数据结构。一个 VMA 表示在进程的虚拟内存中的一个同类区域。

➤ vm_area_struct 结构

- Linux内核中虚拟内存管理的最基本的管理单元是 struct vm_area_struct，它描述的是一段连续的、具有相同访问属性的虚存空间，该虚存空间的大小为物理内存页面的整数倍。

```
#include <linux/mm.h>
```

```
struct vm_area_struct {
```

```
    unsigned long vm_start; //该VMA所覆盖的虚拟地址起始地址
```

```
    unsigned long vm_end; //该VMA所覆盖的虚拟地址结束地址
```

```
    struct file *vm_file; //指向与该区域相关联的file结构指针
```

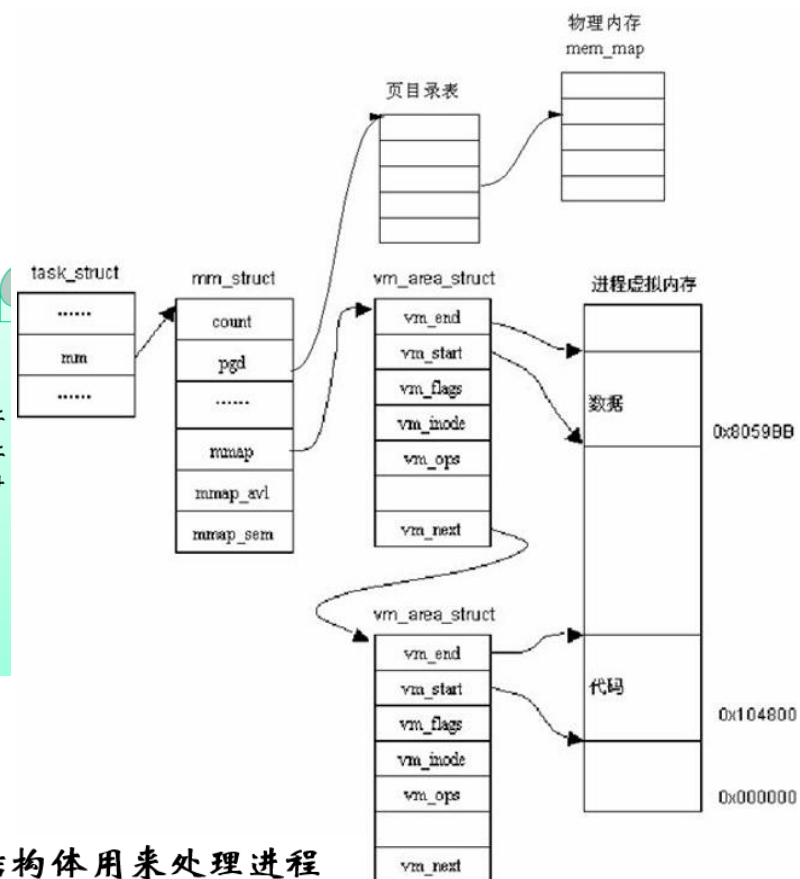
```
    unsigned long vm_pgoff; //以页为单位，文件中该区域的偏移
```

```
    unsigned long vm_flags; //描述该区域的一套标志
```

```
    struct vm_operations_struct *vm_ops; //内核能调用的函数
```

```
    void *vm_private_data; //驱动程序用来保存自身信息的成员
```

```
}
```



➤ vm_operations_struct 结构

- struct vm_operations_struct 结构体用来处理进程的内存需求。

```
#include <linux/mm.h>
```

```
struct vm_operations_struct {
```

```
    //内核调用open函数，以允许实现VMA的子系统初始化该区域
```

```
    void (*open)(struct vm_area_struct * vma);
```

```
    //当销毁一个区域时，内核将调用close操作
```

```
    void (*close)(struct vm_area_struct * vma);
```

```
    //当一个进程访问属于合法VMA的页，但该页又不在内存中时，则为相关区域调用fault函数
```

```
    int (*fault)(struct vm_area_struct *vma, struct vm_fault *vmf);
```

```
}
```

➤ struct vm_fault 结构

- struct vm_fault 结构体用来处理缺页错问题。

```
#include <linux/mm.h>
struct vm_fault {
    unsigned int flags;      /* FAULT_FLAG_xxx flags */
    pgoff_t pgoff;          /* Logical page offset based on vma */
    void __user *virtual_address; /* 产生缺页错的虚拟地址 */
    struct page *page;       /* fault的返回值 */
};
```

➤ Mmap 设备操作

mmap 方法是 file_operations 结构的一部分，并且执行 mmap 系统调用时将调用该方法，实现物理地址空间和虚拟地址空间间的内存映射。

➤ mmap 系统调用

- void* mmap (void * addr , size_t len , int prot , int flags ,int fd , off_t offset)
 - ◆ addr: 指定映射的起始地址，通常设为 NULL，由系统指定。
 - ◆ length: 映射到内存的文件长度。
 - ◆ prot: 映射区的保护方式:PROT_EXEC/PROT_READ/PROT_WRITE
 - ◆ flags: 映射区的特性:MAP_SHARED/MAP_PRIVATE
 - ◆ fd: 由 open 返回的文件描述符，代表要映射的文件。
 - ◆ offset: 以文件开始处的偏移量，必须是分页大小的整数倍，通常为 0，表示从文件头开始映射。

➤ mmap 文件操作

- int (*mmap) (struct file *, struct vm_area_struct *)
- mmap 通过两种方式完成页表的建立，即完成物理地址和虚拟地址之间的映射：
 - ◆ 使用 remap_pfn_range 一次建立所有页表;
 - ◆ 使用 vm fault 方法每次建立一个页表，当用户访问 VMA 中的页，而该页又不在内存中时，将调用相关的 vm fault 函数实现内存映射。

➤ 使用 remap_pfn_range 映射内存

- int remap_pfn_range(struct vm_area_struct *vma, unsigned long virt_addr,unsigned long pfn, unsigned long size, pgprot_t prot)
 - ◆ vma: 虚拟内存区域指针
 - ◆ virt_addr: 虚拟地址的起始值
 - ◆ pfn: 要映射的物理地址所在的物理页帧号，可将物理地址>>PAGE_SHIFT 得到。
 - ◆ size: 要映射的区域的大小。
 - ◆ prot: VMA 的保护属性。

```
static int memdev_mmap(struct file*filp, struct vm_area_struct *vma)
{
    struct mem_dev *dev = filp->private_data; /*获得设备结构体指针*/
    vma->vm_flags |= VM_IO;
    vma->vm_flags |= VM_RESERVED;
```

```

        if(remap_pfn_range(vma,vma->vm_start,virt_to_phys(dev->data)>>PAGE_SHIFT,
vma->vm_end - vma->vm_start, vma->vm_page_prot))
            return -EAGAIN;
        return 0;}

```

➤ 使用 fault 函数映射内存

- 当用户访问 VMA 中的页，而该页又不在内存中时，将调用相关的 fault 函数实现内存映射
- struct page * (*fault)(struct vm_area_struct *vma, struct vm_fault *vmf)
 - ◆ vma: 虚拟内存区域指针
 - ◆ vmf: 引起错误的虚拟地址相关结构体

```

void simple_vma_open(struct vm_area_struct *vma) {
    printk(KERN_NOTICE "Simple VMA open, virt %lx, phys %lx\n", vma->vm_start, vma->vm_pgoff <<
PAGE_SHIFT);
}

void simple_vma_close(struct vm_area_struct *vma) {
    printk(KERN_NOTICE "Simple VMA close\n");
}

int simple_vma_fault(struct vm_area_struct *vma, struct vm_fault *vmf) {
    struct page *pageptr;
    unsigned long baseaddr = ADDRESS; /* 驱动程序申请空间的首地址 */
    unsigned long virtaddr = vmf-> virtual_address - vma->vm_start + baseaddr;

    pageptr = virt_to_page(virtaddr);
    if (!pageptr)
        return VM_FAULT_SIGBUS;
    get_page(pageptr);
    /* got it, now increment the count */
    vmf->page = pageptr;
    printk("Virtual Address %p\n", vmf-> virtual_address);
    return 0;
}

static struct vm_operations_struct simple_fault_vm_ops = {
    .open = simple_vma_open,
    .close = simple_vma_close,
    .fault = simple_vma_fault,
};

static int simple_fault_mmap(struct file *filp, struct vm_area_struct *vma) {
    unsigned long offset = vma->vm_pgoff << PAGE_SHIFT;
    if (offset >= __pa(high_memory) || (filp->f_flags & O_SYNC))
        vma->vm_flags |= VM_IO;
    vma->vm_flags |= VM_RESERVED;
    vma->vm_ops = &simple_fault_vm_ops;
    simple_vma_open(vma);
    return 0;
}

struct file_operations simple_fault_ops={

```



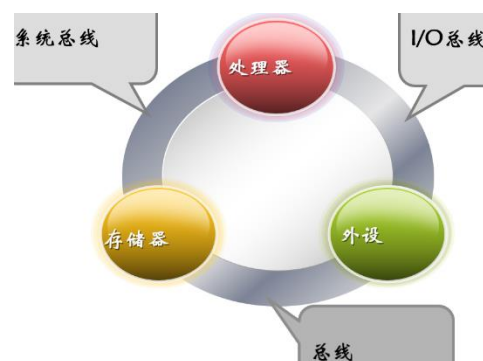
```

.open = simple_open,
.release = simple_release,
.mmap = simple_fault_mmap,
};
int main()
{
    int dev;
    int loop;
    char *ptrdata;

    dev=open(DEVICE_FILENAME,O_RDWR|O_NDELAY);
    ptrdata=(char*)mmap(0,  SHARE_MEM_SIZE,
                        PROT_READ|PROT_WRITE,
                        MAP_SHARED,  dev,  0);
    for(loop=0;loop<SHARE_MEM_PAGE_COUNT;loop++)
    {
        printf("[%d---]\n",*(ptrdata+4096*loop));
    }
    munmap(ptrdata,SHARE_MEM_SIZE);
    close(dev);

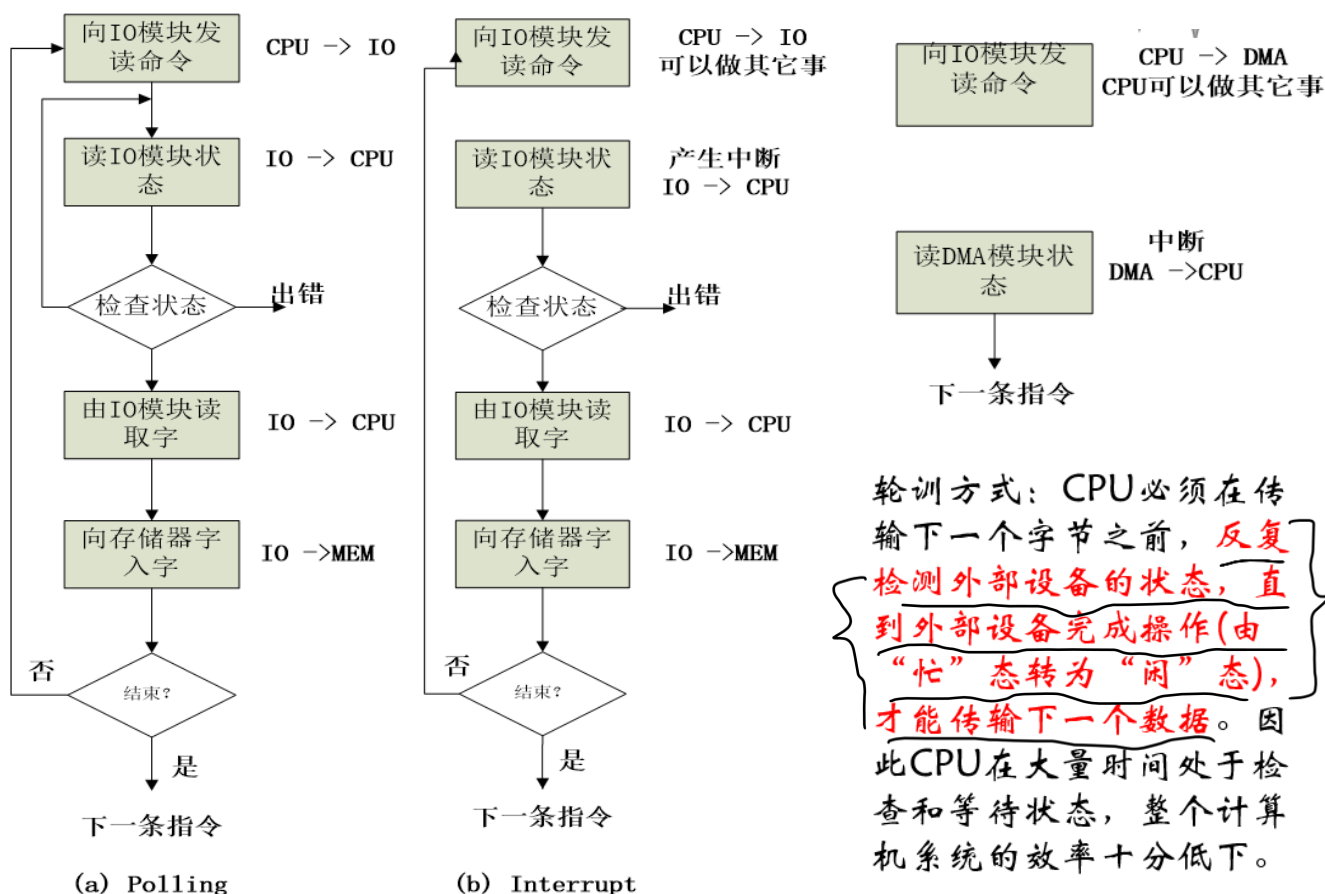
    return 0;
}

```



第09讲 DMA 机制

一、 I/O 体系结构简介



中断方式：

CPU 发送一个 I/O 命令，告诉 I/O 设备，而后转向其它的工作进程，当 I/O 设备接受到来自 CPU 的 I/O 命令时，进行相关的硬件准备工作，当硬件条件得到满足时，则 I/O 设备则生一个外部中断，CPU 响应中断，暂停当前工作，转向去处理与 I/O 设备之间进行交换的数据，数据处理完成继续执行当前工作

DMA 方式：

CPU 向 I/O 设备发一组命令，然后 I/O 设备发起一个 DMA 中断请求信号，如果 CPU 响应，则告知 DMA 控制器，DMA 控制器完成一些准备工作之后向 CPU 发起总线请求信号，如果 CPU 响应，则 DMA 控制器接管系统总线，在相关的控制信息作用下，完成相关的数据传输操作，当操作完成，DMA 则通过产生一个中断告知 CPU 操作已经完成

➤ 为什么采用 DMA 技术

➤ 轮训方式

- CPU 必须在传输下一个字节之前，反复检测外部设备的状态，直到外部设备完成操作(由“忙”态转为“闲”态)，才能传输下一个数据。因此 CPU 在大量时间处于检查和等待状态，整个计算机系统的效率十分低下。

➤ 中断方式

- 仍然占用了 CPU 相当多的时间。这里因为 I/O 设备每传输一个数据字节都要向 CPU 发出一次中断请求，也就要中断一次 CPU 的当前工作。而且 CPU 在每次响应中断之后还要作保护现场操作(把当前各个寄存器中的内容送入内存保存起来)，再调用中断处理程序，执行完毕中断处理程序之后，还要恢复现场(把刚才送入内存保存的内容再重新装入各个寄存器中)，以便继续执行中断之前的工作。这样，在连续传输一个数据块的过程中，CPU 要反复多次被中断，并且花费很多时间去处理中断，使得 CPU 的效率仍然不能得到很好地发挥。整个系统效率的提高仍然受到限制。所以，在传输的数据量很大时，中断方式也不能满足要求。

➤ DMA 方式

- 最明显的一个特点是它不是用软件而是采用一个专门的硬件电路——DMA 控制器（DMAC）来控制内存与外设或是内存与内存之间的数据交流，无须 CPU 介入，大大提高了 CPU 的工作效率。

二、 DMA 基本情况

➤ DMA 概念的界定

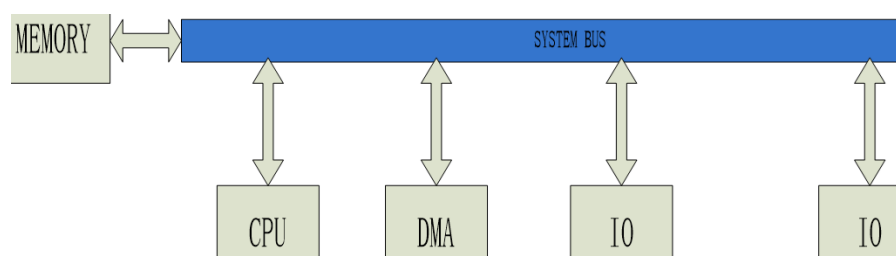
DMA 是 Direct Memory Access 的缩写。其意思是“存储器直接访问”。它是指一种高速的数据传输操作，允许在外部设备和存储器之间直接读写数据，即不通过 CPU，也不需要 CPU 干预。

整个数据传输操作在一个称为“DMA 控制器”的控制下进行的。CPU 除了在数据传输开始和结束时作一点处理外，在传输过程中 CPU 可以进行其它的工作。这样，在大部分时间里，CPU 和输入输出都处在并行操作。因此，使整个计算机系统的效率大大提高。

➤ DMA 结构分析

➤ 单总线、I/O 分离的 DMA 结构

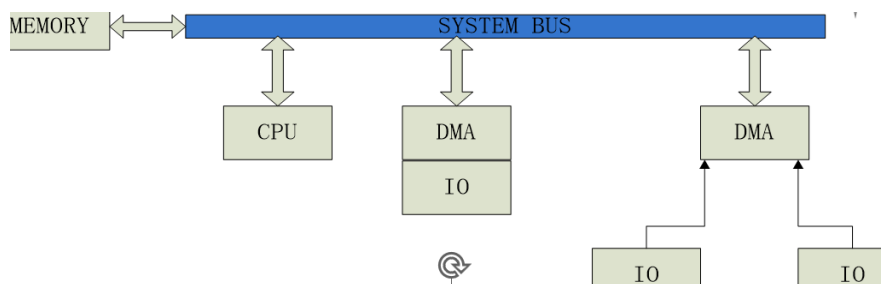
DMA 模块作为 CPU 代理，采用编程 I/O，在存储器和 I/O 模块之间经过 DMA 交换数据，采用共享总线技术



- ✓ 优点：结构简单、配置代价较小、价格便宜；
- ✓ 缺点：这种结构中，存储器和 I/O 模块之间经过 DMA 数据交换的路径是：MEMORY (I/O MODULE) ---- SYSTEM BUS ----DMA ----SYSTEMBUS ----I/O(MEMORY)，可以看出传送一个字需要消耗两个总线周期，传输效率比较低

➤ 单总线、I/O 集成的 DMA 结构

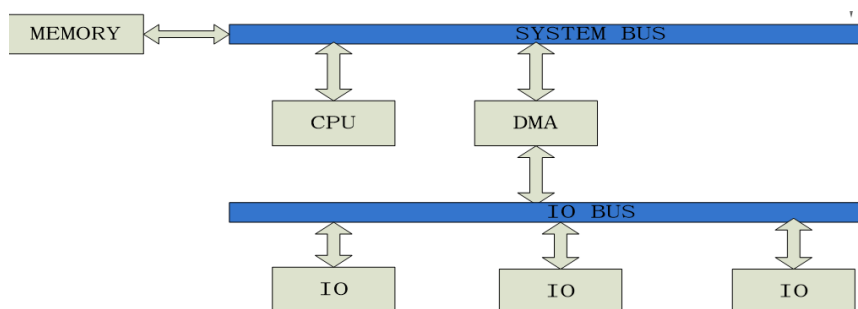
通过集成 DMA 和 I/O 功能，即 DMA 模块和一个或是多个 I/O 模块集成在一起，这样所需总线周期数可以削减



- ✓ 这种结构的优点在于，DMA 逻辑集成为 I/O 模块的一部分，存储器和 I/O 模块之间经 DMA 数据交换路径简化为，MEMORY(I/O)----SYSTEMBUS ---I/O(MEMORY)，传送一个字节只需要一个总线周期，节省了总线周期

➤ I/O 总线的 DMA 结构

这种结构将单总线、I/O 集成的 DMA 结构中的 I/O 模块和 DMA 模块的连接改为 I/O 总线，使集成 DMA 逻辑的概念向前迈进了一步



- ✓ 优点是，通过 I/O 总线连接 I/O 外设与 DMA 模块连接，I/O 外设与 DMA 控制器都可以做成与总线的标准接口，满足了嵌入式微处理器结构可扩展的要求。

➤ 三种 DMA 结构对比

- 以上三种结构具有各自的特点，适用于不同的场合：
 - ◆ 单总线、I/O 分离的 DMA 结构简单，配置代价较小，价格便宜，适合于功能相对简单的微处理器；
 - ◆ 单总线、I/O 集成的 DMA 结构虽然同为单总线结构，但 I/O 外设通过 DMA 模块和总线连接，传输效率高；
 - ◆ I/O 总线的 DMA 结构除了继承了传输效率高的特点外，采用了扩展的双总线结构，适用于 I/O 外设较多的微处理器。

➤ DMA 传输方式分析

周期挪用方式

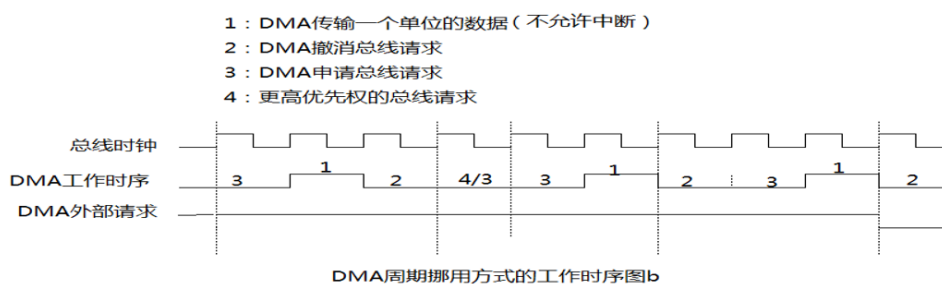
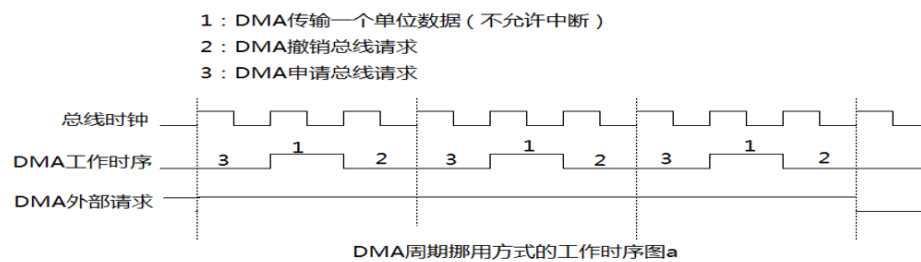
- 指 DMA 每传输完一个单位的数据后，将撤销总线申请，一个周期后再提出总线申请；
 - ◆ 如果总线仲裁逻辑许可 DMA 继续控制总线，由于外部请求依然有效，DMA 将接着传输下一个单位的数据；
 - ◆ 如果因为其它原因总线仲裁逻辑收回 DMA 的总线控制权，DMA 将等到总线控制权有效后才能

接着传输下一个单位的数据。

- ◆ 在正常情况下，周期挪用方式下在整个 DMA 传输过程中外部请求保持有效，总线申请很快变为有效。于是在 DMA 控制器接收到新的应答之后，下一个数据的 DMA 传输即刻开始。这样，在周期挪用方式下，DMA 控制器周而复始地进行“总线请求——总线应答——总线传输——总线释放”的循环，直到指定的数据全部传完。相应地，DMA 控制器的地位也在总线从属性和总线主控性之间交替变化。

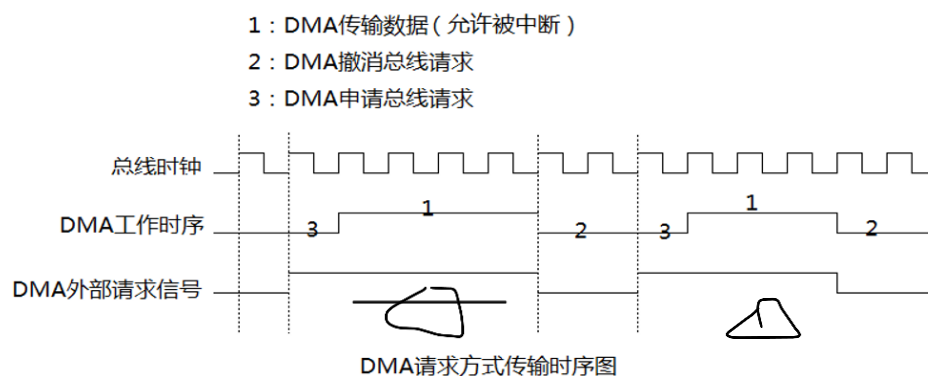
- 由于周期挪用方式在相邻总线申请之间只传输一个字的数据，故也称为单字传输方式。

- ◆ 在单字传输方式下，要实现数据的背对背不间断传输，必须保证在整个传输过程中请求有效且没有更高优先权的总线主控器争夺总线控制权。周期挪用方式便于在 DMA 数据传输过程中插入其它总线操作。



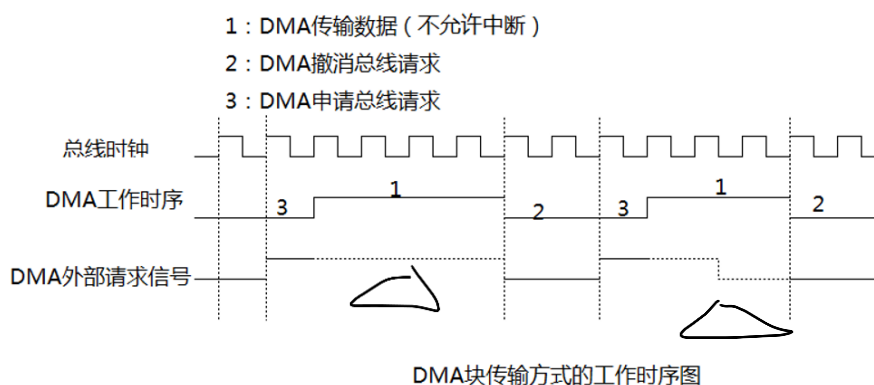
请求方式

- 请求传输方式指在 DMA 传输过程中，(非意外情况下总线仲裁逻辑将不会收回总线控制权)如果外部请求有效 DMA 将保持总线申请，数据得以连续传输；如果外部请求无效 DMA 将撤销总线申请，交还总线控制，等到下一次外部请求有效才发出总线申请，获得许可后继续数据传输。
- 请求传输方式允许请求对象打断 DMA 数据传输过程，去完成特殊处理或进行数据的准备，然后再进行 DMA 请求，请求 DMA 接着被打断的 DMA 过程进行数据传输。
- 请求传输方式也允许在 DMA 传输过程中，总线仲裁逻辑因为(非 DMA 主动撤销总线申请)特殊情况收回总线控制权，但在处理完特殊情况后总线仲裁逻辑应该授予 DMA 总线控制权，让 DMA 完成被打断 DMA 过程。在请求传输方式下，一个完整的数据交换过程可以因为数据交换请求的不连续性分成若干个子过程完成。



块传输方式

- 块传输方式与请求传输方式不同之处在于 DMA 在数据传输过程中，将不理睬外部请求是否有效而保持总线申请，直到数据传送完毕才撤销总线申请，交还总线控制权。
- 请求对象发出一次有效的 DMA 请求以后，在 DMA 传输过程中不必再保持请求有效，因此请求对象无法中断 DMA 传输过程。
- 在块传输方式下，一般不希望总线仲裁逻辑在 DMA 传输过程中收回总线控制权。因为在 DMA 传输过程中请求对象可能不再发出 DMA 请求，如果总线仲裁逻辑收回总线控制权打断 DMA 过程，即使总线仲裁逻辑再授予 DMA 总线控制权，DMA 也将无法接着被中断的 DMA 过程进行数据传输，从而导致数据传输出错。
- 因此，在块传输方式下不能轻易地中断 DMA 传输过程。在块传输方式下，值得注意的是总线仲裁器必须防止一路 DMA 长期占用总线导致其它 DMA 任务阻塞的问题。

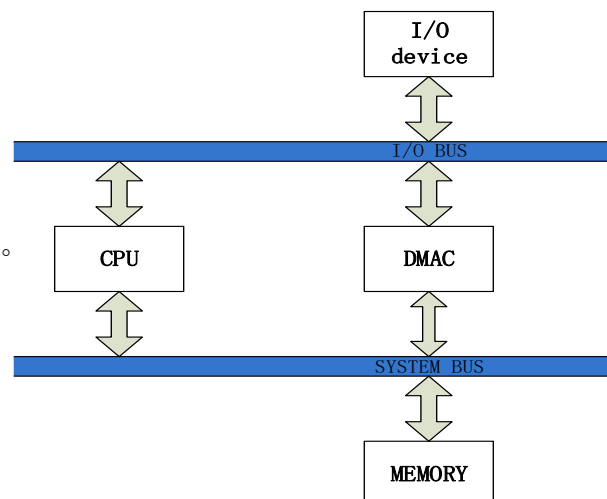


➤ DMA 对数据的操作类型

- 数据检验
- 数据传输与检索并举
- 数据检索
- 数据传输

➤ 典型的 DMA 工作结构

图示是一种比较简单的 DMA 控制方式结构，
“I/O 设备——CPU——内存”是一条数据传输通路，
“I/O 设备——DMAC——内存”显然是另外一条相对独立的通道。
这样，如果 I/O 设备与内存之间有大量的数据需要传输时，
可在 CPU 控制下直接进行，也可以选择通过 DMA 进行。



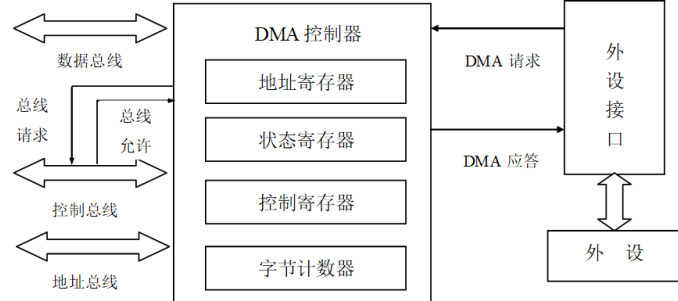
一般来说，DMA 控制器应具备下述基本功能：

- (1) 接收到相应外设发出的就绪态信号后，向 CPU 发出 DMA 请求；
- (2) CPU 响应请求，将总线控制权移交 DMAC；
- (3) DMAC 对内存寻址，执行数据传送；
- (4) 能发出读/写控制信号；
- (5) 能决定传送的字节数，判断 DMA 传送是否结束；
- (6) 发 DMA 操作结束信号给 CPU，释放总线，使 CPU 重新获得总线控制权；

➤ DMA 控制器（DMAC）

DMA 控制器可以像 CPU 那样获得总线的控制权，完成外设与存储器之间的数据高速交换。

DMA 控制器不但要与外设连接，以接受外设发出的 DMA 操作请求和在 DMA 期间对外设进行控制，还要与 CPU 连接，以请求总线的控制权；同时，它还需要与三大总线连接，以便进行总线的控制。



地址寄存器：包括源地址和目的地址寄存器。

- 在进行 DMA 操作之前，在 CPU 控制下将源地址和目的地址分别装入 DMAC 的源地址和目的地址寄存器；
- 在进入 DMA 操作后，由这些地址寄存器提供出源地址和目的地址，并在传送数据的同时，由硬件以加 1 或减 1 来修改地址寄存器的值。

控制/状态寄存器：控制寄存器用于选择 DMAC 的操作类型、工作方式、传送方向和有关参数。

- 这种选择是通过 CPU 在 DMA 操作之前向控制寄存器写入相应的控制字来实现的。
- 状态寄存器用于寄存 DMA 传送前后的状态。
- DMA 传送结束后，CPU 通过对该寄存器执行输入指令即可读入状态字，了解所需的状态和结果

字节计数器：用于控制传送数据块的长度。

- 在进入 DMA 操作之前，CPU 将数据块长度(字节数)装入字节计数器中。
- 进入 DMA 操作后，每传送一个字节数据，由硬件自动修改计数器的值(减)；当计数器溢出时，便使 DMA 方式的数据传送结束。

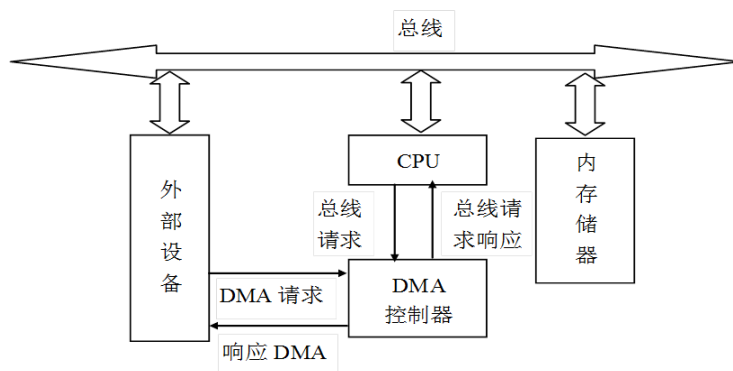
总线接口和总线控制逻辑：

- 这部分电路主要用于在 DMA 传送之前接受来自 CPU 的控制字和根据外部/内部 DMA 请求向 CPU 转发总线请求；
- DMA 操作期间进行定时和发出读写控制信号；DMA 操作结束后向 CPU 发出中断申请和状态信息。

➤ DMA 控制原理

在 DMA 数据传输过程中脱离了 CPU 的控制，采用 DMA 控制器来管理和控制数据传输的整个过程，而 DMA 控制器的启动初始化，都是由 CPU 控制完成。

DMA 控制原理图如下：



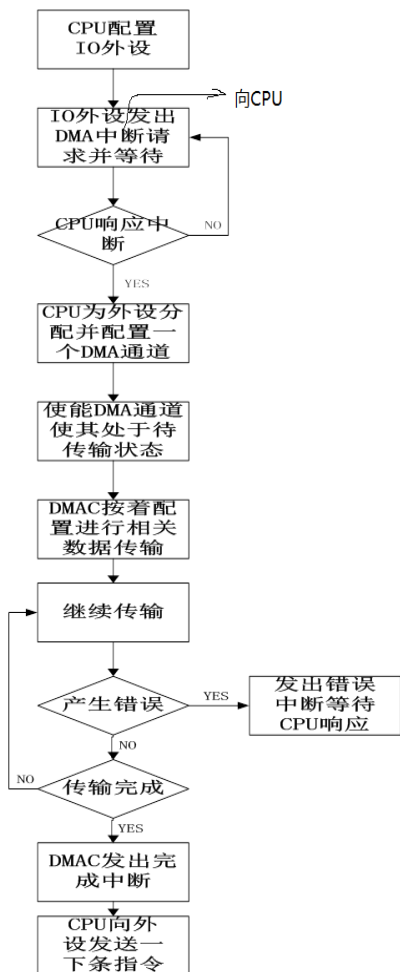
➤ DMA 工作流程

整个工作流程包括两个方面：

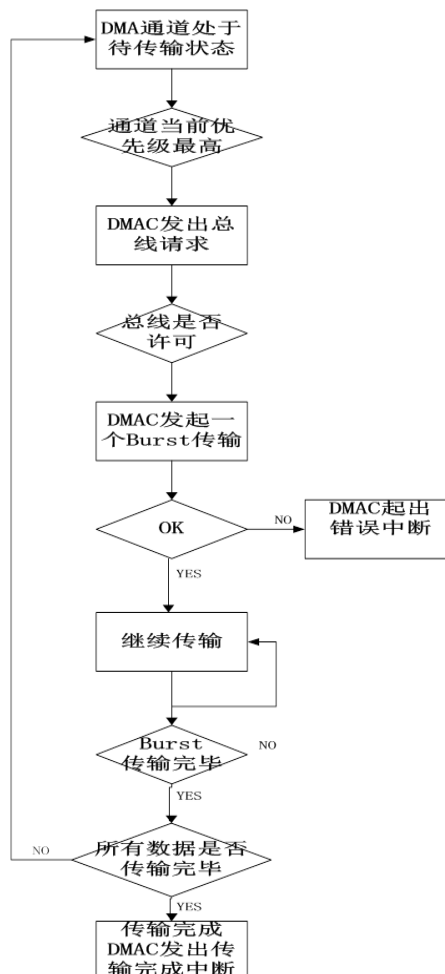
CPU 工作部分；DMA 硬件工作部分；

如果更加详细的去划分各自的工作范围，我们可以按着一个任务体从开始到结束的顺序来划分，划分结果如下：

- 1、CPU 配置 I/O 设备，使其向 CPU 申请 DMA 功能，也就是申请一个 DMA 通道；
- 2、根据任务需求，配置申请的 DMA 通道和建立一个任务实体；
- 3、提交任务实体，从硬件的角度来说，就是通过底层硬件写相关寄存器的值，比如源、目的地址 register、传输数据量等，从软件的角度来说，就是将任务实体加入到 DMA 任务队列中；
- 4、CPU 将总线控制权移交给 DMAC，而后发起 DMA 传输；
- 5、传输完毕或是在传输过程中遇到错误，DMA 分别产生一个完成中断或错误中断，提示 CPU 进行相关的数据处理；
- 6、CPU 有选择性的响应 5 中所提到的中断，此时挂起 DMA 去处理相关任务；处理完毕如果还有任务体没有被执行则重新启动 DMA，转到 3；
- 7、任务结束，结果输出；



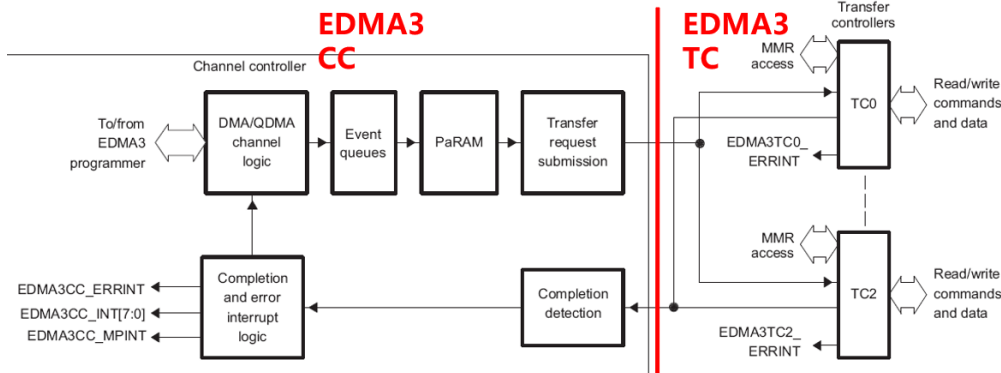
(a) CPU委托DMAC为IO外设进行DMA传输流程



(b) DMAC具体操作流程

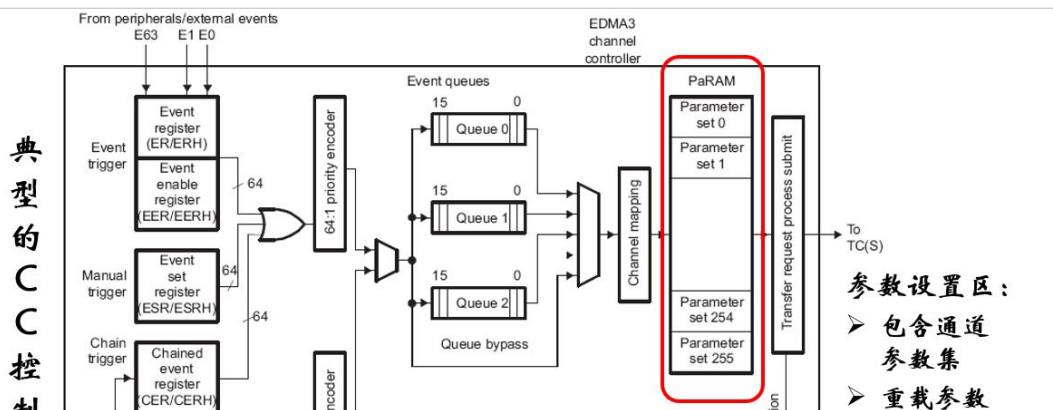
三、 EDMA (Enhanced DMA)

- EDMA 是数字信号处理器(DSP)中用于快速数据交换的重要技术，具有独立于 CPU 的后台批量数据传输的能力，能够满足实时图像处理中高速数据传输的要求。
- EDMA 典型的应用包括：
 - 1：软件驱动的数据传输
 - 2：外设事件驱动的数据传输
 - 3：数据排序或者抽取

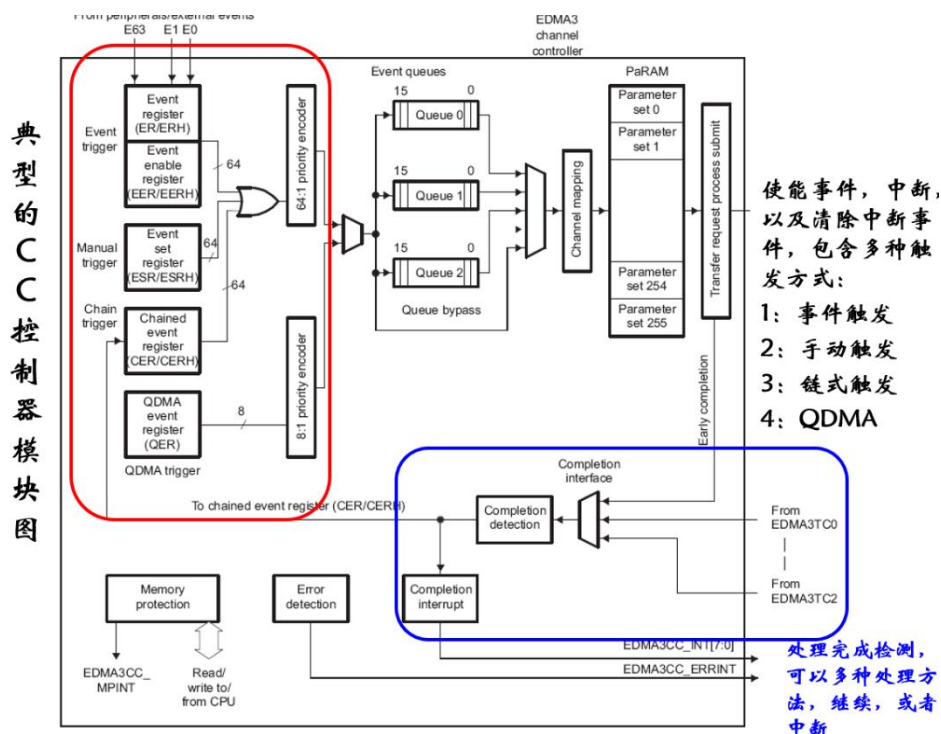
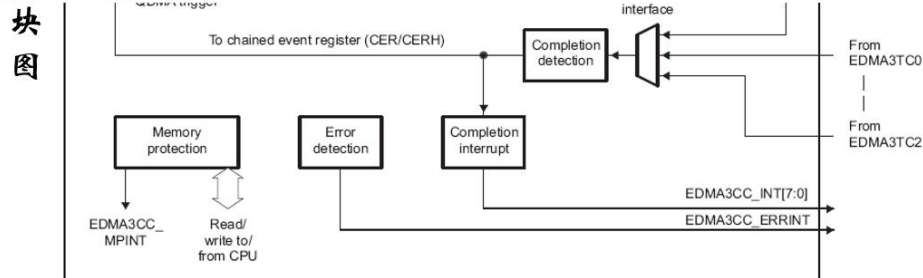


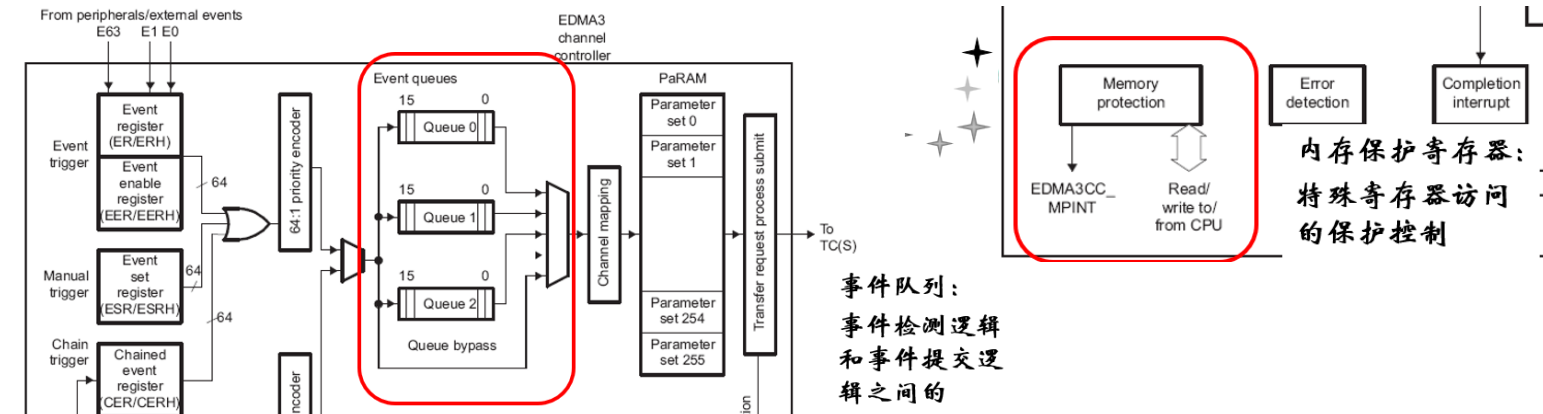
EDMA3 CC作为整个控制器的用户接口，包含了参数RAM区，通道控制逻辑，中断控制寄存器，向TC发送传输请求

EDMA3 TC负责真正的数据传输，进行具体内存的读写，对用户而言是透明的



Parameter RAM (PaRAM): The PaRAM maintains parameter sets for channel and reload parameter sets. You must write the PaRAM with the transfer context for the desired channels and link parameter sets. EDMA3CC processes sets based on a trigger event and submits a transfer request (TR) to the transfer controller.





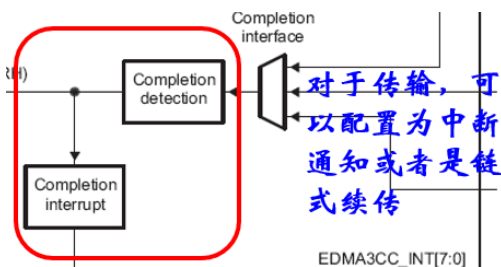
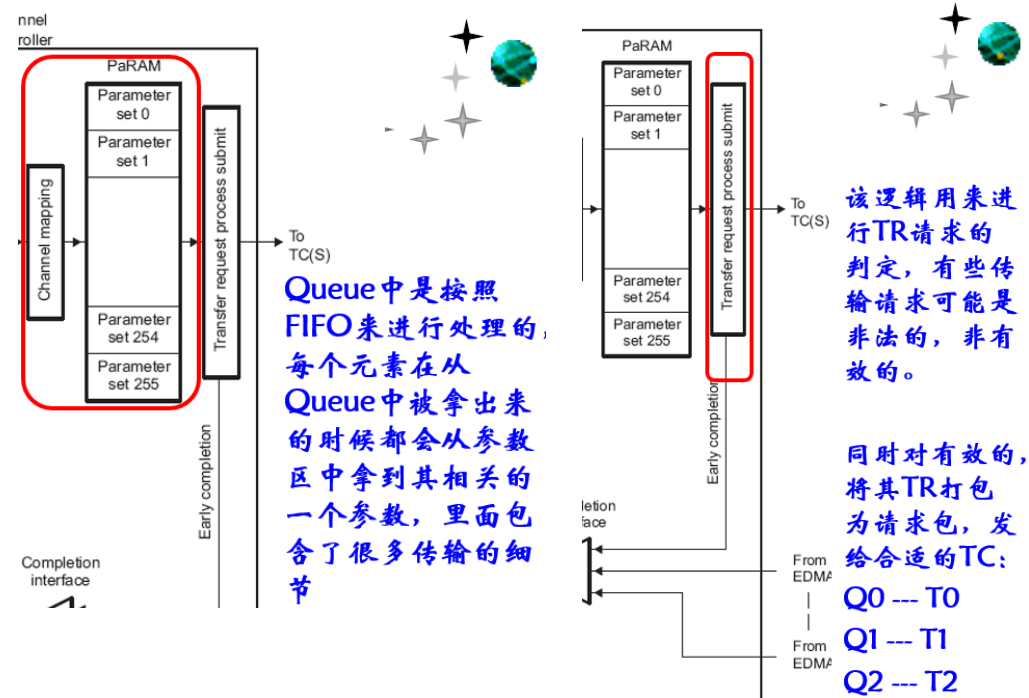
Event queues: Event queues form the interface between the event detection logic and the transfer request submission logic.

➤ 典型的 CC 控制器事件处理流程

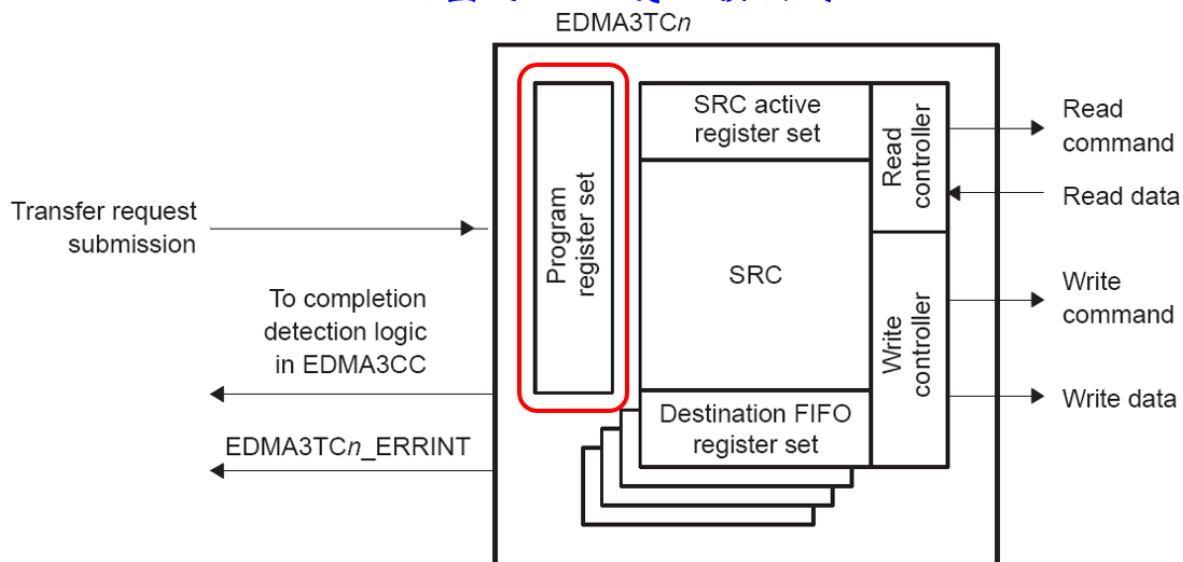
基本从模块图是从左向右，包含了几个步骤：

- 1: 事件产生：
 - 1.外部触发 2.手动产生 3.链表产生 4.QDMA 生成
- 2: 队列管理
- 3: 事件提交
- 4: 事件响应

事件一旦被识别，就会被塞入到 Queue 中去



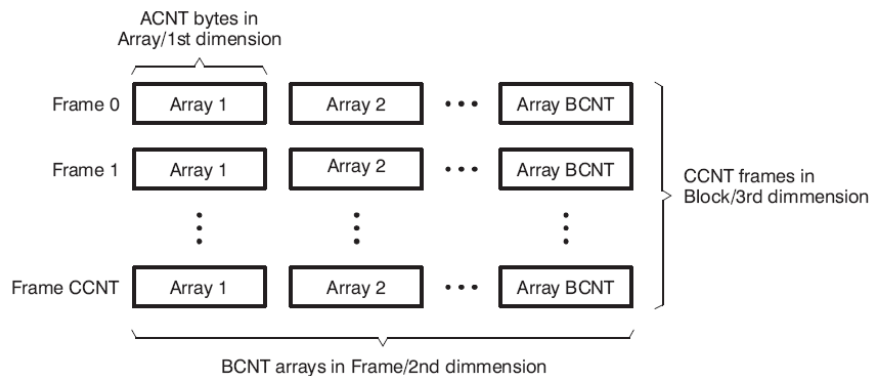
典型的TC控制器模块图



程序寄存器设置，实际存放的就是TRP

➤ EDMA3 的数据传输分类

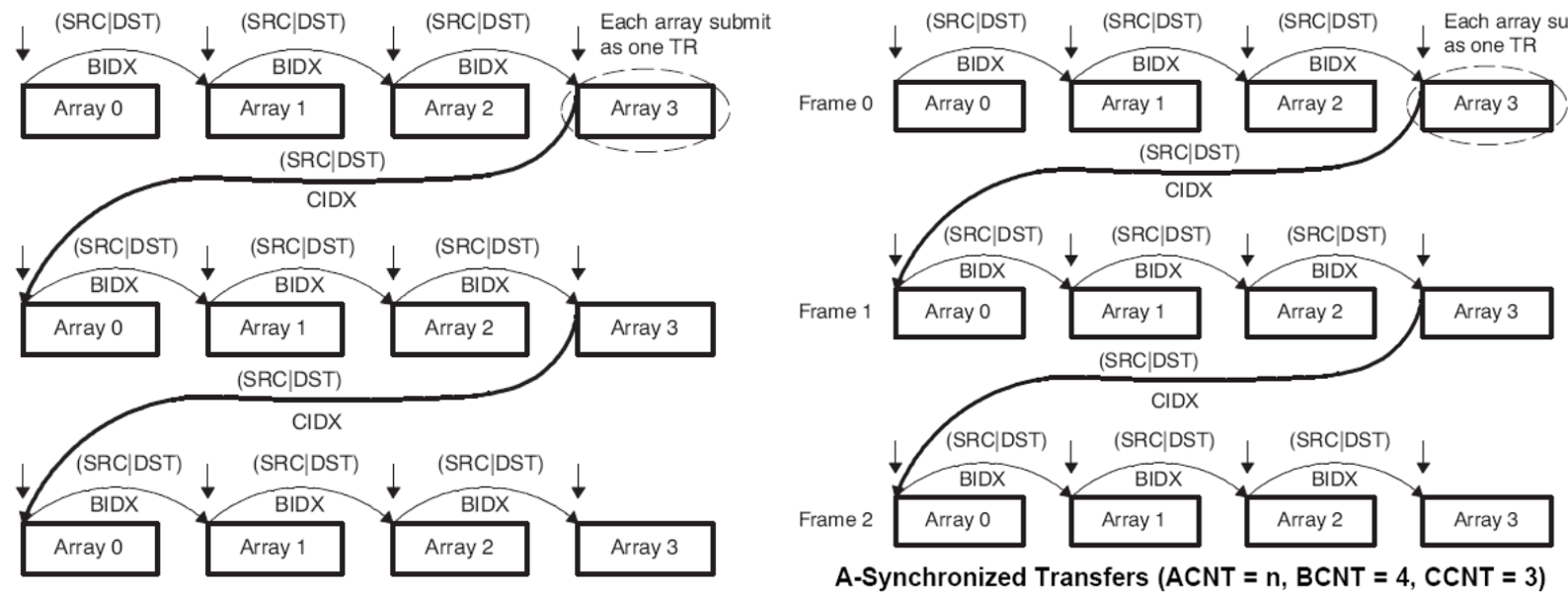
- 1: 一维 DMA
- 2: 二维 DMA
- 3: 三维 DMA



1. 第一维度或数组 (A):
传输中的第一维度由 ACNT 个连续字节组成。
2. 第二维度或帧 (B):
传输中的第二维度由 BCNT 个 ACNT 字节数组构成。
第二维度中的每个数组传输之间通过使用 SRCBIDX 或 DSTBIDX 编程的索引进行分隔。
3. 第三维度或块 (C):
传输中的第三维度由 CCNT 个 BCNT 数组的 ACNT 字节帧构成。
第三维度中的每个传输之间通过使用 SRCCIDX 或 DSTCIDX 编程的索引进行分隔。

数组索引与地址计算说明

1. 数组分隔机制:
所有数组均通过 SRCBIDX (源缓冲区间隔) 和 DSTBIDX (目标缓冲区间隔) 实现物理分隔
如图所示，相邻数组的地址空间通过索引值实现非连续排列
2. 地址递推关系:
数组 N 的起始地址 = 数组(N-1)的起始地址 + SRCBIDX (源端操作)
或 = 数组(N-1)的起始地址 + DSTBIDX (目标端操作)
该机制确保在数据传输过程中各数组保持预设的存储间距
3. BIDX 参数特性:
索引值以字节为单位进行编程设定
支持正向和负向地址偏移配置
实现多维数组的跨步访问控制



A-Sync（一维同步）传输协议

事件触发机制

1. 传输粒度

- 每个 EDMA3 同步事件发起 ACNT 字节的第一维度传输
- 即：单事件/事务包（TR Packet）仅承载单个数组（ACNT 字节）的传输信息

2. 事件资源需求

$$Total\ Events = BCNT \times CCNT$$

- 需消耗 BCNT×CCNT 个事件完成整个 PaRAM 参数集的传输服务
- 事件触发逻辑关系：
单事件 → 单数组 → 多事件 → 参数集服务完成

技术特性

- **硬件限制**：TR Packet 字段宽度限制单事件传输信息容量
- **适用场景**：低带宽传感器数据采集（如 ADC 单通道采样）

AB-Sync（二维同步）传输协议

帧传输优化

1. 多维传输能力

- 单 EDMA3 同步事件触发二维传输（即完整帧传输）
- 单 TR Packet 可承载 BCNT 个数组（每个 ACNT 字节）的帧信息

2. 事件效率提升

$$Total\ Events = CCNT$$

- 仅需 CCNT 个事件即可完成相同 PaRAM 参数集服务
- 触发逻辑升级为：
单事件 → 多数组(帧) → 少事件 → 参数集服务完成

➤ EDMA3 的参数区内容及管理

参数为基础的 DMA 管理，内部存放的是各个 DMA 通道需要使用到的参数。
可以看做是一个表。
同时表可以被分割为对应的集合。

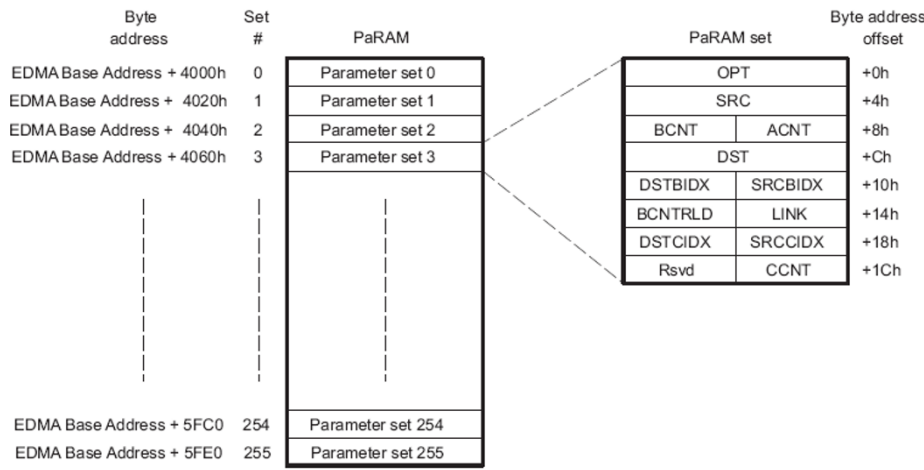
总共包含 256 个参数集合

➤ EDMA3 通道参数表核心配置解析

(按偏移地址排序)

0h - OPT (通道选项寄存器)

- **功能**：控制传输模式与数据对齐特性
- **关键位域**：
 - 传输类型 (单发/突发)
 - 地址递增模式 (固定/线性/步进)
 - 数据位宽 (8/16/32 位)
- **技术限制**：需与物理总线位宽匹配



4h - SRC (通道源地址)

- **地址要求**：需按数据元素大小对齐 (如 32 位数据需 4 字节对齐)
- **动态更新**：每次传输后按 SRCBIDX 值偏移

8h - ACNT (第一维度计数)

- **物理意义**：单次传输的连续数据块长度
- **取值范围**：1-65535 (单位：字节)
- **应用场景**：定义 DMA 每次搬运的"数据包"大小

Ch - DST (通道目标地址)

- **地址约束**：必须匹配目标存储器的物理映射特性
- **优化技巧**：配合 DSTBIDX 实现环形缓冲区管理

10h - BCNT (第二维度计数)

- **多维控制**：定义 ACNT 数据包的传输次数
- **重载机制**：通过 BCNTRLD 实现循环传输 (视频帧缓冲典型应用)
- **数学关系**：总传输量 = ACNT × BCNT

SRCBIDX/DSTBIDX (二维索引)

- **跨步控制**：
 - 正索引：存储器地址递增 (常规阵列访问)
 - 负索引：实现滑动窗口算法 (卷积神经网络常用)
- **动态调整**：支持运行时修改实现弹性缓冲区

14h - LINK (链接地址)

- **链式 DMA**：参数集耗尽后自动加载新配置
- **安全机制**：FFFFh 表示传输终止 (防指针越界)

BCNTRLD (BCNT 重载值)

- **循环特性**：当 BCNT 递减至 0 时自动重载
- **模式依赖**：仅在 A-Sync 模式下生效 (二维传输自包含)

18h - SRCCIDX/DSTCIDX (第三维索引寄存器组)

SRCCIDX (源第三维索引)

- **数值范围**：有符号 16 位整数 ($-32768 \leq \text{value} \leq 32767$)
- **传输模式行为**：
 - **A-Sync 模式**：
帧末数组首地址 → 下帧首地址偏移 (适合非连续存储跳跃)
偏移量 = 下一帧首地址 - 当前帧末地址
 - **AB-Sync 模式**：

帧首数组首地址 → 下帧首地址偏移 (适合连续存储跳跃)
偏移量 = 下一帧首地址 - 当前帧首地址

DSTCIDX (目标第三维索引)

- 符号特性: 同 SRCCIDX
- 模式差异:
 - A-Sync: 基于帧末地址的跨帧跳变 (类似链表结构)
 - AB-Sync: 基于帧首地址的跨帧跳变 (类似数组结构)

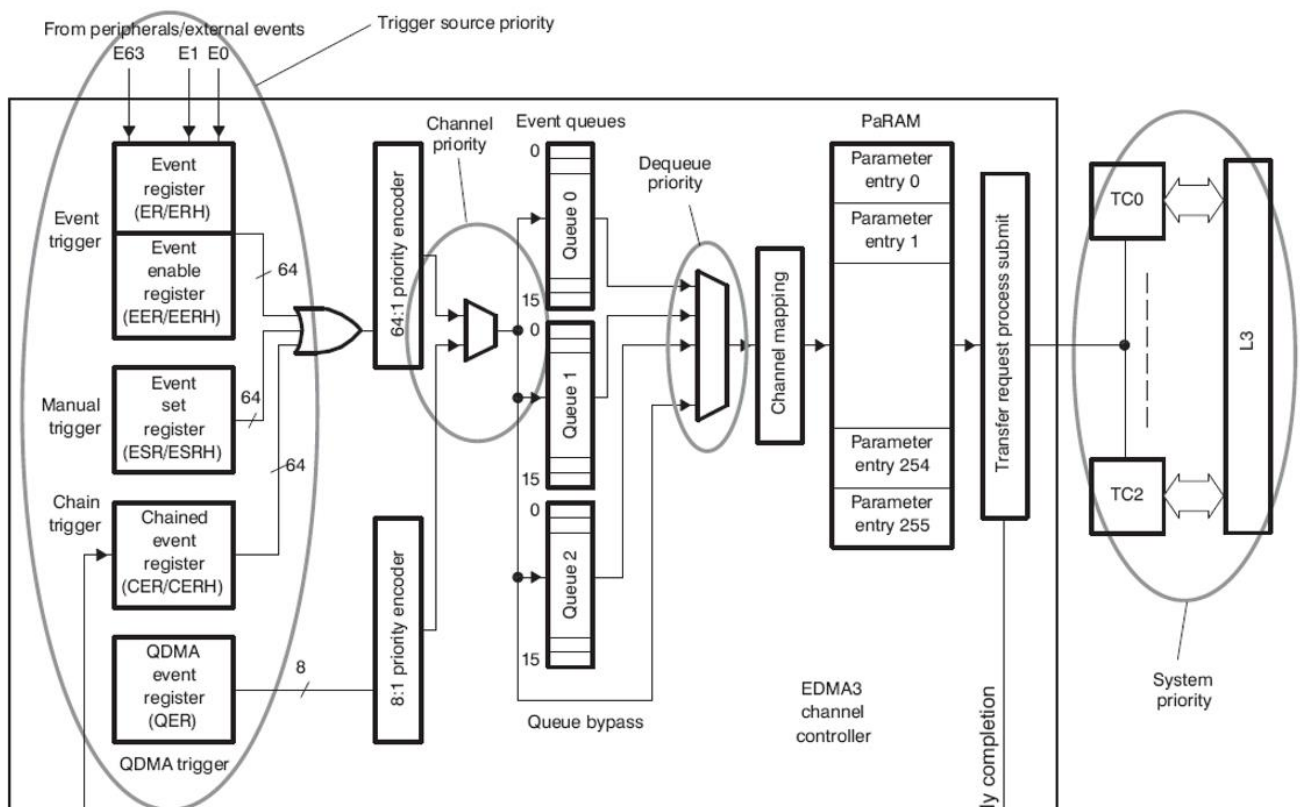
1Ch - CCNT (第三维计数)

- 数值约束: 无符号 16 位整数 ($1 \leq \text{value} \leq 65535$)
- 物理意义: 块包含的帧数 (每帧 = BCNT×ACNT 字节)
- 动态行为:
 - 传输时自动递减, 支持多块级联传输
 - 与 BCNTRLD 配合实现无限循环传输 (需配置 LINK 地址)
- 传输量公式: $\text{Total_Transfer} = \text{ACNT} \times \text{BCNT} \times \text{CCNT}$

RSVD (保留位)

- 硬件约束:
 - 必须强制写入 0x0
 - 写 0x1 可能触发总线保护机制 (TI C6000 系列典型行为)

EDMA3 的优先级



EDMA3 的块数据搬移

当一维DMA

ACNT = 256

BCNT = 1 CCNT = 1

Channel Source Address (SRC)

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	...	...	
...	...	...	244	245	246	247	248
249	250	251	252	253	254	255	256

Channel Destination Address (DST)

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	...	...	
...	...	...	244	245	246	247	248
249	250	251	252	253	254	255	256

ACNT: 单个元素字节数

BCNT: 每帧包含的

帧数 (1-行/层数)

CCNT: 每帧所包含的

数 (几行/层数)

Parameter Contents

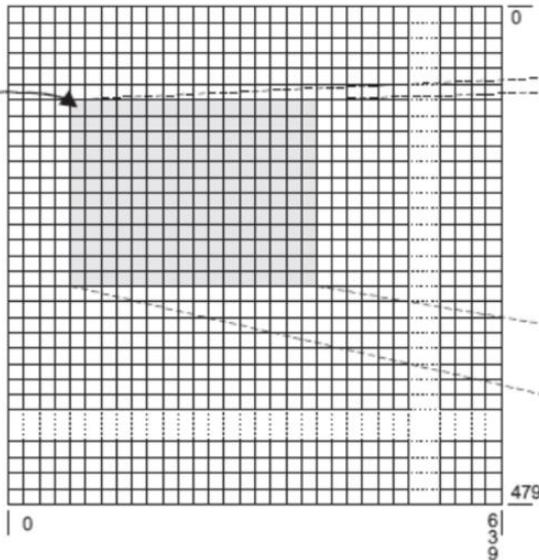
0010 0008h	
Channel Source Address (SRC)	
0001h	0100h 256
Channel Destination Address (DST)	
0000h	0000h
0000h	FFFFh
0000h	0000h
0000h	0001h

Parameter

Channel Options Parameter (OPT)	
Channel Source Address (SRC)	
Count for 2nd Dimension (BCNT)	Count for 1st Dimension (ACNT)
Channel Destination Address (DST)	
Destination BCNT Index (DSTBIDX)	Source BCNT Index (SRCBIDX)
BCNT Reload (BCNTRLD)	Link Address (LINK)
Destination CCNT Index (DSTCIDX)	Source CCNT Index (SRCCIDX)
Reserved	Count for 3rd Dimension (CCNT)

EDMA3 的抽出部分数据的搬移

Channel Source Address (SRC)



Channel Destination Address (DST)

0_1	0_2	0_3	0_4	0_5	0_6	0_7	0_8	0_9	0_A	0_B	0_C	0_D	0_E	0_F	0_10
1_1	1_2	1_3	1_4	1_5	1_6	1_7	1_8	1_9	1_A	1_B	1_C	1_D	1_E	1_F	1_10
2_1	2_2	2_3	2_4	2_5	2_6	2_7	2_8	2_9	2_A	2_B	2_C	2_D	2_E	2_F	2_10
3_1	3_2	3_3	3_4	3_5	3_6	3_7	3_8	3_9	3_A	3_B	3_C	3_D	3_E	3_F	3_10
4_1	4_2	4_3	4_4	4_5	4_6	4_7	4_8	4_9	4_A	4_B	4_C	4_D	4_E	4_F	4_10
5_1	5_2	5_3	5_4	5_5	5_6	5_7	5_8	5_9	5_A	5_B	5_C	5_D	5_E	5_F	5_10
6_1	6_2	6_3	6_4	6_5	6_6	6_7	6_8	6_9	6_A	6_B	6_C	6_D	6_E	6_F	6_10
7_1	7_2	7_3	7_4	7_5	7_6	7_7	7_8	7_9	7_A	7_B	7_C	7_D	7_E	7_F	7_10
8_1	8_2	8_3	8_4	8_5	8_6	8_7	8_8	8_9	8_A	8_B	8_C	8_D	8_E	8_F	8_10
9_1	9_2	9_3	9_4	9_5	9_6	9_7	9_8	9_9	9_A	9_B	9_C	9_D	9_E	9_F	9_10
A_1	A_2	A_3	A_4	A_5	A_6	A_7	A_8	A_9	A_A	A_B	A_C	A_D	A_E	A_F	A_10
B_1	B_2	B_3	B_4	B_5	B_6	B_7	B_8	B_9	B_A	B_B	B_C	B_D	B_E	B_F	B_10

Parameter Contents

0010 000Ch	
Channel Source Address (SRC)	
000Ch 12个/行	0020h 256/8 = 4 Byte/行
Channel Destination Address (DST)	
0020h	0500h
0000h	FFFFh
0000h	0000h
0000h	0001h 1行

Parameter

Channel Options Parameter (OPT)	
Channel Source Address (SRC)	
Count for 2nd Dimension (BCNT)	Count for 1st Dimension (ACNT)
Channel Destination Address (DST)	
Destination BCNT Index (DSTBIDX)	Source BCNT Index (SRCBIDX)
BCNT Reload (BCNTRLD)	Link Address (LINK)
Destination CCNT Index (DSTCIDX)	Source CCNT Index (SRCCIDX)
Reserved	Count for 3rd Dimension (CCNT)

EDMA3 的数据排序的搬移



Channel Source Address (SRC)

主要做
他呀

列

1024行

A_1	A_2	A_3	...	...	A_1022	A_1023	A_1024
B_1	B_2	B_3	...	...	B_1022	B_1023	B_1024
C_1	C_2	C_3	...	...	C_1022	C_1023	C_1024
D_1	D_2	D_3	...	...	D_1022	D_1023	D_1024

BCNT=1024

Channel Destination Address (DST)

做转置

16B

A_1	B_1	C_1	D_1
A_2	B_2	C_2	D_2
A_3	B_3	C_3	D_3
...	...	...	...
...	...	...	...
A_1022	B_1022	C_1022	D_1022
A_1023	B_1023	C_1023	D_1023
A_1024	B_1024	C_1024	D_1024

Parameter Contents

0090 0004h	
Channel Source Address (SRC)	
0400h	BCNT ACNT 0004h 每个元素4B
Channel Destination Address (DST)	
0010h 16B步	0004h
0000h	FFFFh
0004h 4字节偏移	1000h 1024x4B 12字节偏移
0000h	0004h CCNT

Parameter

Channel Options Parameter (OPT)	
Channel Source Address (SRC)	
Count for 2nd Dimension (BCNT)	Count for 1st Dimension (ACNT)
Channel Destination Address (DST)	
Destination BCNT Index (DSTBIDX) 偏移 4x4Byte = 4x4x8 = 16x8	Source BCNT Index (SRCBIDX)
BCNT Reload (BCNTRLD)	Link Address (LINK)
Destination CCNT Index (DSTCIDX)	Source CCNT Index (SRCCIDX)
Reserved	Count for 3rd Dimension (CCNT)

实际上，ABCD 分别是一个数组，

ACNT:数组中每个元素的大小

BCNT:每个数组一共多少个元素

CCNT:一共多少个数组

SRCBIDX/DSTBIDX (二维索引):

表示对于一个数组，每个元素的偏移量（从一个元素的首地址到下一个元素首地址）

SRCBIDX: 在源的一个数组里，到数组中的下一个元素的偏移为 SRCBIDX

DSTBIDX: 在目的的一个数组里，到数组的下一个元素的偏移为 DSTBIDX

SRCCIDX/DSTCIDX (三维索引):

表示对于一组数据，每个数组首地址的偏移量

SRCCIDX: 在源中，一个数组到下一个数组（首地址）的偏移为 SRCCIDX

DSTCIDX: 在目的中，一个数组到下一个数组（首地址）的偏移为 DSTCIDX